Supercompilation as Cyclic Proof Search for the Calculus of Constructions

Dr. Gavin Mendel-Gleason
Scidonia Limited
scidonia.ai

May 4, 2026

Abstract

This paper presents the first formulation of supercompilation for a dependently typed programming language, establishing a precise correspondence between supercompilation and cyclic sequent proof search for the Calculus of Constructions (CoC) extended with inductive types. To the best of our knowledge, this is the first work to expose the exact operational connection between supercompilation and proof search at the level of individual inference rules.

We show that supercompilation operations—driving (lazy unfolding), case-splitting on neutral terms, and folding (memoization)—correspond exactly to inference rules in a focused cyclic sequent calculus. The asynchronous phase of the sequent calculus captures deterministic driving steps, while the synchronous phase captures the essential choice points of supercompilation: case analysis and fold-point selection. Cyclic backlinks in the proof graph correspond precisely to memo table lookups in the supercompiler, and the global trace condition ensuring soundness of cyclic proofs corresponds to the termination criterion based on structural descent in supercompilation.

This correspondence is formalized as a four-claim pipeline: (1) supercompilation always yields a locally-valid pre-proof, (2) with the trace-condition check it yields a ranked cyclic proof via a budget trace construction, (3) the residualised program is proved CIU-equivalent to the source via fuel induction, and (4) the pipeline is validated on concrete examples. All four claims are mechanized in Coq. The framework provides a principled foundation for supercompilation in dependently typed languages and opens new avenues for proof-directed program optimization.

Contents

1 Introduction

Supercompilation is a semantics-driven program transformation technique based on symbolic evaluation (driving), constructor case analysis, generalisation, and fold/refold via memoisation. Despite decades of development for functional languages, there is no widely accepted exposition of supercompilation for a dependently typed core language, where types depend on terms and where transformation steps must preserve typing and observational meaning.

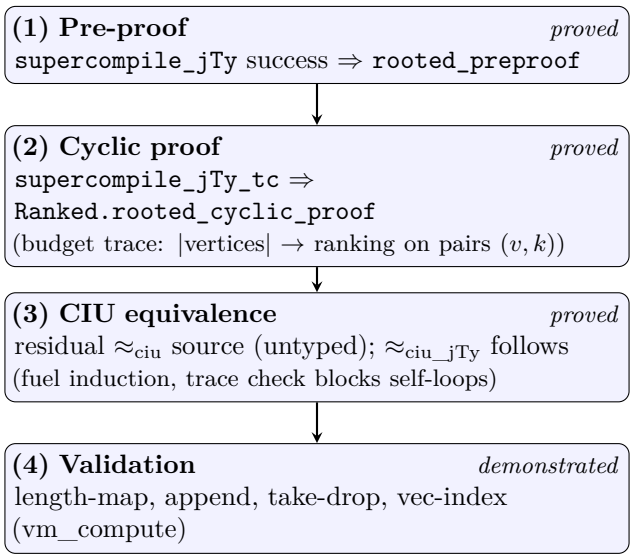
This paper proposes a proof-theoretic account: *supercompilation can be presented as focused cyclic sequent proof search* for a CoC-style dependent calculus with inductives. The correspondence is meant to be exact at the level of operational moves: driving steps are invertible (asynchronous) sequent steps, case-splitting is a synchronous rule, and folding is cyclic closure via back-links.

Contributions.

- A concrete CoC-with-inductives term calculus recap (Section 2).
- A focused cyclic sequent calculus designed to mirror supercompilation moves (Section 3).
- A precise operational dictionary between drive/split/fold and local sequent steps (Section 4).
- A proof-engineering perspective on *post-hoc* induction principles: cyclic derivations allow the induction scheme to emerge from the discovered cycle structure (Section 1.3).
- A mechanisation status report for the Coq development (Section 8).

Outline. Section 1.1 states the central thesis as a pipeline of four claims. Sections 2–3 define the source calculus and the focused cyclic sequent system. Section 4 presents worked correspondences. Section 6 discusses related work, and Section 7 gives the formal equivalence statements.

1.1 Main result pipeline



The four claims form a dependency chain: each tier builds on the previous. All are mechanized in Coq with full proofs (0 axioms, 0 admitted lemmas).

Claim 1: Supercompilation yields a pre-proof. When the supercompiler succeeds on a well-typed input, the resulting configuration graph directly forms a *rooted pre-proof*. Every vertex is labelled with a judgement and satisfies its local sequent rule; every successor is a valid vertex; and the edge structure corresponds exactly to the drive/split/fold operations of the supercompiler. This claim is **mechanized** in Coq via `all_supercompiled_programs_yield_preproof` (`SupercompilationCorrespondence.v:1659`) and the three graph-level correspondence lemmas (`drive_corresponds_to_async_edge`, `split_corresponds_to_sync_edge`, `memo_corresponds_to_fold`). See Section 7 for the formal statement (Theorem 7.1).

Claim 2: Termination yields a cyclic proof. A pre-proof becomes a *cyclic proof* when it additionally satisfies a global progress condition: every cycle must contain at least one progress edge (a case-split on a neutral scrutinee), and there exists a well-founded ranking that strictly decreases along progress edges. The supercompiler enforces this via a trace-condition check during graph construction. We prove this by constructing a budget-instrumented trace graph: vertices are lifted to pairs (v, k) where $k \in [0, B]$ is a budget counter consumed on progress edges and preserved on non-progress edges. The resulting trace graph is acyclic, yielding a full ranking_condition with rank $\lambda(v, k).\, k$. This claim is **mechanized**: `supercompile_yields_rooted_cyclic_proof` (`SupercompilationCorrespondence.v`). See Section 7 for the formal statement (Theorem 7.4).

Claim 3: Cyclic proofs imply CIU equivalence. The ultimate soundness guarantee is *contextual equivalence* (CIU): the program read off from a valid cyclic proof should be observationally indistinguishable from the original source program under all closing substitutions. This claim is **mechanized**: `supercompile_ciu_soundness_untyped` and `supercompile_ciu_soundness_typed` (`SupercompilationC`

by fuel induction. The trace condition prevents self-loops that would introduce non-termination. See Section 7 for the formal statement (Theorem 7.5).

Claim 4: The pipeline obtains for our examples. The framework is validated on a suite of standard supercompilation examples (length-map fusion, append length normalisation, take-drop, etc.). For each example, the supercompiler produces a residual, and computational equality with the expected result is verified by `vm_compute` in Coq (`Equiv/SupercompileChecklistIndexPipeline.v`). See Section 7 for the formal statement (Proposition 7.6) and Section 5 for a detailed walkthrough.

1.2 Why focusing matters

Focusing separates proof search into an *asynchronous* (invertible) phase and a *synchronous* (choice-bearing) phase. This mirrors supercompilation practice:

- Driving is largely deterministic (invertible phase).
- Generalisation and folding are the essential choice points (synchronous phase).
- Case-splitting sits at the boundary: it is forced when the scrutinee is neutral, but introduces branching.

1.3 Why cyclic proofs matter: post-hoc induction principles

A crucial advantage of the cyclic proof framework is that it allows the *induction principle to emerge during proof search* rather than requiring it to be specified upfront. This is fundamentally different from traditional inductive proof systems.

Traditional induction: In standard natural deduction or sequent calculus for inductive types, the induction principle must be chosen at the start of the proof. For example, to prove a property about lists, one typically applies structural induction on the list, which commits to a specific recursive structure immediately. If the property requires a different induction scheme (e.g., on a different variable, or on a measure combining multiple arguments), the proof fails and must be restarted with a new principle.

Cyclic induction: In the cyclic proof setting, we instead:

1. Begin with the goal sequent (no induction principle required)
2. Drive and split as needed, following the term structure
3. Form backlinks when we encounter goals that match previous vertices
4. The induction principle is *read off* from the completed cyclic proof graph

The key insight is that the cycle structure itself *is* the induction principle. The trace condition (ensuring structural descent through progress events) guarantees soundness, but the actual induction scheme—which variables are analyzed, in what order, and with what invariants—emerges naturally from the proof search process.

Connection to supercompilation: This matches exactly how supercompilation works: the residual program's recursive structure is discovered by the driving process, not prescribed in advance. When the supercompiler folds a configuration back to an earlier one (via memo table lookup), it has implicitly discovered a valid recursion scheme.

Example: Consider proving that `length (map f l)` equals `length l`. A traditional proof requires choosing to induct on l at the outset. In the cyclic/supercompilation setting, after driving and splitting we encounter a subgoal `length (map f xs)` which matches the original goal under substitution $l \mapsto xs$, yielding a backlink and thus an induction scheme discovered post-hoc.

2 Term calculus (CoC + inductives)

This section presents the term calculus used throughout the paper: a Calculus of Constructions (CoC) extended with inductive types and fixpoints. This is the source language for both the natural deduction typing system and the sequent calculus developed in Section 3.

2.1 Syntax

Terms are defined inductively as follows:

$$\begin{array}{rcl} t, A, B & ::= & x \mid \mathsf{Sort}\, i \mid \Pi(A).\, B \mid \lambda(A).\, t \mid t\, u \\ & \mid & \mathsf{fix}(A).\, t \mid \mathsf{Ind}^I(\vec{a}) \mid \mathsf{roll}^I_c(\vec{a}) \mid \mathsf{case}^I(t; C; \vec{b}) \end{array}$$

where:

- x ranges over de Bruijn indices (variables)
- $\mathsf{Sort}\, i$ is the universe at level i (we use a cumulative hierarchy)
- $\Pi(A).\, B$ is the dependent function type with domain A and codomain B (binding one variable)
- $\lambda(A).\, t$ is a lambda abstraction with explicit type annotation A
- $t\, u$ is function application
- $\mathsf{fix}(A).\, t$ is a fixpoint with type annotation A (binding one variable for recursion)
- $\mathsf{Ind}^I(\vec{a})$ is an applied inductive type I with parameters/indices $\vec{a}$
- $\mathsf{roll}^I_c(\vec{a})$ is a constructor application (constructor c of inductive I)
- $\mathsf{case}^I(t; C; \vec{b})$ is dependent pattern matching on scrutinee t with motive C and branches $\vec{b}$

Conventions: We write $t[u/x]$ for capture-avoiding substitution and $t[u/]$ for substitution under one binder (replacing de Bruijn index 0). Contexts Γ are lists of types. The global environment Σ records inductive definitions (constructors, elimination rules, strict positivity).

2.2 Natural deduction typing rules

The typing judgement $\Sigma; \Gamma \vdash t : A$ states that term t has type A in context Γ under global environment Σ. We abbreviate this as $\Gamma \vdash t : A$ when Σ is clear from context.

2.2.1 Structural rules

$$\frac{x : A \in \Gamma}{\Gamma \vdash x : A}\ (\textsc{Var})$$

$$\frac{\Gamma \vdash t : A \quad \Gamma \vdash B : \mathsf{Sort}\, i}{\Gamma, B \vdash t : A}\ (\textsc{Weak})$$

2.2.2 Universe and conversion

$$\frac{i < j}{\Gamma \vdash \mathsf{Sort}\, i : \mathsf{Sort}\, j} \text{ (Sort)}$$

$$\frac{\Gamma \vdash t : A \quad \Gamma \vdash B : \mathsf{Sort}\, i \quad A \equiv_\beta B}{\Gamma \vdash t : B} \text{ (Conv)}$$

Here $A \equiv_\beta B$ denotes β-convertibility (the reflexive-symmetric-transitive closure of β-reduction).

2.2.3 Dependent functions (Π-types)

$$\frac{\Gamma \vdash A : \mathsf{Sort}\, i \quad \Gamma, A \vdash B : \mathsf{Sort}\, j}{\Gamma \vdash \Pi(A).\, B : \mathsf{Sort}\ \max(i, j)} \text{ (Pi-Form)}$$

$$\frac{\Gamma \vdash \Pi(A).\, B : \mathsf{Sort}\, i \quad \Gamma, A \vdash t : B}{\Gamma \vdash \lambda(A).\, t : \Pi(A).\, B} \text{ (Pi-Intro)}$$

$$\frac{\Gamma \vdash t : \Pi(A).\, B \quad \Gamma \vdash u : A}{\Gamma \vdash t\, u : B[u/]} \text{ (Pi-Elim)}$$

2.2.4 Fixpoints (recursion)

$$\frac{\Gamma \vdash A : \mathsf{Sort}\, i \quad \Gamma, A \vdash t : A}{\Gamma \vdash \mathsf{fix}(A).\, t : A} \text{ (Fix-Form)}$$

The fixpoint $\mathsf{fix}(A).\, t$ represents a recursive definition where the recursive call is represented by de Bruijn index 0 under the binder.

2.2.5 Inductive types

Let Σ define an inductive type I with constructors $\{c_0, \ldots, c_{n-1}\}$. For each constructor c, we have a type schema T_c recorded in Σ.

$$\frac{\Sigma(I) = (\vec{p}, i\vec{d}x, \ldots) \quad \Gamma \vdash \vec{a} : \vec{\tau}}{\Gamma \vdash \mathsf{Ind}^I(\vec{a}) : \mathsf{Sort}\, i} \text{ (Ind-Form)}$$

$$\frac{\Sigma(I) = (\ldots, c : T_c, \ldots) \quad \Gamma \vdash \vec{a} : T_c[\vec{p}/]}{\Gamma \vdash \mathsf{roll}^I_c(\vec{a}) : \mathsf{Ind}^I(\vec{p})} \text{ (Ind-Intro)}$$

2.2.6 Pattern matching (elimination)

$$\frac{\begin{array}{l}\Gamma \vdash t : \mathsf{Ind}^I(\vec{a}) \\ \Gamma, \mathsf{Ind}^I(\vec{a}) \vdash C : \mathsf{Sort}\, j \\ \forall c \in \mathrm{constrs}(I),\ \Gamma \vdash b_c : T_c \to C[\mathsf{roll}^I_c(\cdots)/]\end{array}}{\Gamma \vdash \mathsf{case}^I(t; C; \vec{b}) : C[t/]} \text{ (Ind-Elim)}$$

The motive C is a type former depending on the scrutinee. Each branch b_c handles constructor c, and the overall case expression has type $C[t/]$ (the motive instantiated at the scrutinee).

2.3 Operational semantics

We use call-by-name (CBN) reduction, which is essential for the supercompilation correspondence (driving = lazy unfolding).

2.3.1 Reduction rules

$$\begin{aligned}
(\lambda(A).\, t)\, u &\to t[u/] \qquad (\beta\text{-reduction}) \\
\mathsf{fix}(A).\, t &\to t[\mathsf{fix}(A).\, t/] \qquad (\text{fixpoint unfolding}) \\
\mathsf{case}^I((\mathsf{roll}^I_c(\vec{a})); C; \vec{b}) &\to b_c\, \vec{a} \qquad (\iota\text{-reduction})
\end{aligned}$$

CBN reduces at the outermost (head) position: we contract a β-redex only when the function is a λ, we unfold a fixpoint only at the head, and we perform ι-reduction only when the scrutinee is a constructor. We do *not* reduce under binders in the operational semantics. (Independently, the typing rules use the standard conversion rule based on β-convertibility.)

2.4 Observational equivalence

Two terms are observationally equivalent if they produce indistinguishable results in all closing contexts. We use the CIU (Closed Instantiations of Uses) theorem from the mechanization: terms are equivalent iff they pass the same observations at the same types.

For this paper, the key observation predicate is:

$$\mathsf{Obs}_{\mathbf{N}}(t) \iff t \to^* \mathsf{roll}^{\mathbf{N}}_{\mathsf{zero}}() \ \vee\ t \to^* \mathsf{roll}^{\mathbf{N}}_{\mathsf{succ}}(n) \text{ for some } n$$

Supercompilation preserves observational equivalence: the residual program produced by the supercompiler is observationally equivalent to the input.

3 Focused cyclic sequent calculus

This section presents a cyclic sequent calculus for the term language of Section 2. The calculus is designed to mirror supercompilation operations: *driving* (asynchronous unfolding), *splitting* (synchronous case analysis), and *folding* (cyclic backlinks).

3.1 Sequents and judgements

A sequent has the form $\Gamma \vdash t : A$, representing the goal of finding a term t of type A in context Γ. The sequent calculus manipulates these goals using inference rules.

We use three kinds of judgements:

- **Typing judgements** $\mathsf{jTy}(\Gamma, t, A)$: prove that $t : A$ in context Γ
- **Observation judgements** $\mathsf{jObs}(\Gamma, t, \phi)$: prove that term t satisfies observation ϕ
- **Substitution judgements** $\mathsf{jSub}(\Delta, v, \Gamma)$: witness that vertex v can substitute for context Δ in Γ

3.2 Proof graphs

A *cyclic proof* is a finite directed graph where:

- Each vertex v is labeled with a sequent $\mathsf{label}(v)$
- Each vertex has a list of successor vertices $\mathsf{succ}(v)$ (premises)
- Each vertex has an attached local rule $\mathsf{Rule}(v)$ validating the step
- The graph may contain cycles (back-edges)

A proof graph is *valid* if:

1. Every vertex's local rule is valid (correct premises for the conclusion)
2. Every infinite path contains a *progress event* (a synchronous split, see Section 3.8)

3.3 Asynchronous rules (driving)

These rules correspond to CBN reduction steps and type-directed decomposition. They are *deterministic* and *invertible*: there is no choice in applying them.

3.3.1 β-reduction

$$\frac{\Gamma \vdash t[u/] : A}{\Gamma \vdash (\lambda(B).\, t)\, u : A} \quad (\textsc{drive-beta})$$

3.3.2 Fixpoint unfolding

$$\frac{\Gamma \vdash t[\mathsf{fix}(A).\, t/] : A}{\Gamma \vdash \mathsf{fix}(A).\, t : A} \quad (\textsc{drive-fix})$$

3.3.3 Iota reduction (case-of-constructor)

$$\frac{\Gamma \vdash b_c\, \vec{a} : C[\mathsf{roll}^I_c(\vec{a})/]}{\Gamma \vdash \mathsf{case}^I((\mathsf{roll}^I_c(\vec{a})); C; \vec{b}) : C[\mathsf{roll}^I_c(\vec{a})/]} \quad (\textsc{drive-iota})$$

3.3.4 Commuting conversions

When the scrutinee of a case is not a constructor, we attempt to drive it:

$$\frac{\Gamma \vdash t' : \mathsf{Ind}^I(\vec{a}) \qquad \Gamma \vdash \mathsf{case}^I(t'; C; \vec{b}) : C[t'/]}{\Gamma \vdash \mathsf{case}^I(t; C; \vec{b}) : C[t/]} \quad (\textsc{drive-case-scrut})$$

where $t \to t'$ is a CBN reduction step.

3.4 Synchronous rules (splitting)

These rules introduce *choice*: they analyze a neutral term (variable, stuck case) by case-splitting on its constructors. This is the only source of nondeterminism in proof search.

3.4.1 Case split on neutral variable

$$\frac{\Gamma \vdash b_0 : C[\mathsf{roll}_0^I()/] \qquad \cdots \qquad \Gamma \vdash b_{n-1} : C[\mathsf{roll}_{n-1}^I()/]}{\Gamma, x : \mathsf{Ind}^I(\vec{a}) \vdash \mathsf{case}^I(x; C; \vec{b}) : C[x/]} \quad (\textsc{split-case-var})$$

Each premise corresponds to one constructor of the inductive type. The context in each premise is extended with the constructor arguments.

Key property: This is a *progress event*. In the cyclic proof, every cycle must pass through at least one split rule. This ensures termination of the proof search (see Section 3.8).

3.5 Observation rules

Observation rules prove that a term satisfies a given observation predicate. For natural numbers, the observations are "reduces to zero" and "reduces to successor".

3.5.1 Observe zero

$$\frac{}{\Gamma \vdash \mathsf{roll}_{\mathsf{zero}}^{\mathbf{N}}() : \mathsf{Obs}_{\mathsf{zero}}} \quad (\textsc{obs-zero})$$

3.5.2 Observe successor

$$\frac{\Gamma \vdash n : \mathbf{N}}{\Gamma \vdash \mathsf{roll}_{\mathsf{succ}}^{\mathbf{N}}(n) : \mathsf{Obs}_{\mathsf{succ}}} \quad (\textsc{obs-succ})$$

3.5.3 Drive to observation

$$\frac{\Gamma \vdash t' : \phi}{\Gamma \vdash t : \phi} \quad (\textsc{obs-drive})$$

where $t \to t'$ is a CBN reduction step. This allows driving a term until it reaches an observable form.

3.6 Backlink rules (folding)

A backlink closes a proof goal by pointing to a previous vertex with a compatible sequent. This is the cyclic aspect of the calculus, and it embodies a fundamental difference from traditional inductive proofs: *the induction principle emerges from the cycle structure rather than being prescribed upfront.*

In traditional induction, one must choose an induction scheme before beginning the proof (e.g., "induct on list l"). In cyclic proofs, the backlink itself *is* the induction hypothesis, discovered during proof search when we recognize that a current goal is an instance of a previous one. The cycle formed by the backlink implicitly defines a recursive proof structure, and the trace condition ensures this recursion is well-founded.

3.6.1 Exact match

$$\frac{}{\Gamma \vdash t : A} \quad (\textsc{back-exact})$$

If there exists a prior vertex v' with label $\Gamma \vdash t : A$ (syntactically identical), we can immediately close the goal with a backlink to v'.

3.6.2 Instance match

$$\frac{\mathsf{jSub}(\Delta, v', \Gamma)}{\Gamma \vdash t[\sigma/] : A[\sigma/]} \quad (\textsc{back-inst})$$

If there exists a prior vertex v' with label $\Delta \vdash t : A$ and a substitution $\sigma : \Delta \to \Gamma$, we can close the current goal. The substitution judgement $\mathsf{jSub}(\Delta, v', \Gamma)$ witnesses that σ is a valid instantiation.

Discovering induction post-hoc: The instance match rule is particularly powerful because it allows the proof to discover the appropriate instantiation during search. For example:

- Starting goal: $l : \mathsf{List} \vdash \mathsf{length}\,(\mathsf{map}\,f\,l) : \mathbf{N}$
- After driving and splitting on l, in the `cons` branch we reach: $xs : \mathsf{List} \vdash \mathsf{length}\,(\mathsf{map}\,f\,xs) : \mathbf{N}$
- This matches the original goal with substitution $l \mapsto xs$
- The backlink implicitly defines structural induction on l, discovered post-hoc

This flexibility allows cyclic proofs to find induction schemes that would be difficult to specify in advance, especially for mutual recursion, nested induction, or induction on complex measures.

3.7 Phases and focusing

We partition the rules into two phases:

- **Asynchronous phase**: Apply driving rules eagerly (no backtracking needed)
- **Synchronous phase**: Apply split or backlink (requires choice)

This discipline ensures that:

1. Driving (asynchronous) is always deterministic and eager
2. Splitting (synchronous) is the only choice point
3. Backlinks can be tried at any synchronous phase

This matches the supercompilation strategy exactly: drive terms until stuck, then split on neutrals or fold via memo table.

3.8 Global progress condition

A cyclic proof is *sound* if every infinite path through the graph contains infinitely many progress events. In our setting:

- A **progress event** is a synchronous split rule (SPLIT-CASE-VAR)
- This ensures structural descent: each split analyzes a smaller inductive component

Formally, we attach a *trace state* τ to each vertex, tracking which inductive variable is being analyzed:

$$\tau ::= \mathsf{None} \mid \mathsf{Some}(I, x)$$

The trace updates as follows:

- Driving rules: $\tau' = \tau$ (no change)

- Split rule on variable x of inductive I: $\tau' = \mathsf{Some}(I, x)$
- Backlink: $\tau' = \tau$ (inherit from target)

We define a well-founded order on trace states:

$$\mathsf{Some}(I, x') <_\tau \mathsf{Some}(I, x) \iff x' \text{ is a recursive argument of } x$$

A cycle is valid if the trace state strictly decreases on at least one back-edge. This is equivalent to requiring a split event in the cycle, because splits introduce recursive subterms.

3.9 Relation to supercompilation

The sequent calculus rules directly correspond to supercompilation operations:

- **Driving rules** $\leftrightarrow$ `drive_cbn_once` (CBN unfolding)
- **Split rule** $\leftrightarrow$ `split_case_var` (case analysis on neutral)
- **Backlink** $\leftrightarrow$ memo table lookup (folding)
- **Progress condition** $\leftrightarrow$ termination via structural descent

The bisimulation theorem (Section 7) makes this correspondence precise: every supercompilation step corresponds to exactly one sequent rule application, and vice versa.

3.10 Example: `length (map f l)`

Consider the fusion of `length (map f l)`. The sequent proof proceeds as follows:

1. Start: $l : \mathsf{List} \vdash \mathsf{length}\,(\mathsf{map}\,f\,l) : \mathbf{N}$
2. Drive (unfold map): $\vdash \mathsf{length}\,(\mathsf{case}^{\mathsf{List}}(l; \ldots; \ldots)) : \mathbf{N}$
3. Drive (commute case): $\vdash \mathsf{case}^{\mathsf{List}}(l; \ldots; \ldots) : \mathbf{N}$
4. Split on l:
   - Branch `nil`: $\vdash \mathsf{length}\,\mathsf{nil} : \mathbf{N}$ (drive to zero)
   - Branch `cons`: $x : A, xs : \mathsf{List} \vdash \mathsf{length}\,(\mathsf{map}\,f\,(\mathsf{cons}\,x\,xs)) : \mathbf{N}$
5. In cons branch, drive produces: $\vdash \mathsf{succ}\,(\mathsf{length}\,(\mathsf{map}\,f\,xs)) : \mathbf{N}$
6. Backlink to step 1 (with substitution $l \mapsto xs$)

The cycle passes through the split at step 4, which is a progress event. The trace state tracks descent on l (from l to xs), validating the cycle.

3.11 Vertex classification and correspondence

The connection between the supercompiler's configuration graph and the focused sequent calculus is established by classifying each vertex according to its successor structure. For a vertex v with label $\mathsf{jTy}(\Gamma, t, A)$ and successors $\mathsf{succ}(v) = [w_1, \ldots, w_n]$:

Successors	Rule	SC operation	Phase
$n = 0$	DR-LEAF	leaf (axiom)	—
$n = 1$, $\mathsf{drive\_cbn\_once}(t) \neq t$	DR-CBN-ONCE	driving (async)	invertible
$n = 1$, $\mathsf{drive\_cbn\_once}(t) = t$	DR-FOLD	folding (memo hit)	cyclic closure
$n > 1$	DR-SPLIT	case-split (sync)	choice point

The focused discipline emerges naturally: asynchronous (driving) rules are deterministic and can be applied eagerly; synchronous (split) rules introduce nondeterminism at case boundaries; fold rules close cycles via backlinks to previously seen vertices.

Proposition 3.1 (Vertex classification — mechanized)**.** *For every vertex v in the configuration graph produced by successful supercompilation, if* $\mathsf{cb\_label}(v) = \mathsf{jTy}(\Gamma, t, A)$ *and* $\mathsf{cb\_succ}(v) = [w_1, \ldots, w_n]$*, then the vertex satisfies exactly one of the four rules above. This is proved in Coq by* `cfg_vertex_drive_rule` *(*`SupercompilationCorrespondence.v`*).*

4 Supercompilation as focused proof search

We give the intended dictionary between standard supercompilation concepts and focused cyclic proof search.

4.1 Configurations as sequents

A supercompiler configuration (term + context + type/motive) is represented as a sequent goal node. The supercompiler's configuration graph is then a proof-search graph.

4.2 Driving = invertible sequent steps

Driving decomposes the goal by deterministic computation-like rules (β, iota, commuting conversions). In focusing, these correspond to invertible/asynchronous rule applications.

4.3 Case splitting = left rules on neutral scrutinees

When the scrutinee is neutral, supercompilation propagates information by splitting into one successor per constructor. In a sequent system this is a left rule whose premises correspond to constructor cases.

4.4 Folding/memoisation = cyclic closure

Folding corresponds to closing a goal by back-linking to a previously seen companion sequent, recording the instantiation (explicit substitution arguments).

4.5 Generalisation = stream selection + kernel check

Generalisation introduces a more general sequent that subsumes the current goal and a previous goal, enabling folding. In proof search we treat this as a *stream* of candidate lemma schemata, together with a deterministic validator.

Concretely, for a current sequent goal S and a chosen companion S_0, we assume a stream of candidates $\mathsf{gen\_stream}(S, S_0)$. A single generalisation step consists of:

1. selecting the next candidate $(S_{\mathsf{gen}}, \sigma, \sigma_0)$ from the stream (a synchronous choice), and
2. checking that $S = \sigma(S_{\mathsf{gen}})$ and $S_0 = \sigma_0(S_{\mathsf{gen}})$, plus any required typing/progress side conditions (deterministic).

This presentation is intentionally agnostic about how the stream is produced: it may come from bounded anti-unification, hand-crafted heuristics, or an external oracle (including an LLM), as long as the kernel checks are complete.

4.6 Concrete correspondence examples

We demonstrate the exact correspondence between supercompilation moves and sequent proof steps through worked examples. Each example shows the SC state, the sequent proof state, and why they are the same operation.

Example 1: Fix-unfolding (async driving) Supercompiler move:

Config: $\Gamma \vdash (\mathbf{fix}\ f.\ \lambda x.\ body)\ arg : A$

Drive step: `drive_cbn_once` unfolds fix

Result: $\Gamma \vdash (\lambda x.\ body)[\mathbf{fix}\ f.\ \lambda x.\ body/f]\ arg : A$

Sequent proof step:

$$\frac{\Gamma \vdash (\lambda x.\ body)[\mathbf{fix}\ f.\ \lambda x.\ body/f]\ arg : A}{\Gamma \vdash (\mathbf{fix}\ f.\ \lambda x.\ body)\ arg : A}\ \texttt{dc\_fix}$$

Why they're the same: The SC configuration graph adds a directed edge from the fix node to the unfolded term. The proof graph adds the same edge, justified by the `dc_fix` rule from `drive_cbn_onceR`. Both represent the same computational step (fix-unfolding), and `drive_cbn_once_sound` proves they compute the same result.

Example 2: β-reduction (async driving) Supercompiler move:

Config: $\Gamma \vdash (\lambda x : \tau.\ t)\ u : B$

Drive step: `drive_cbn_once` performs β-reduction

Result: $\Gamma \vdash t[u/x] : B$

Sequent proof step:

$$\frac{\Gamma \vdash t[u/x] : B}{\Gamma \vdash (\lambda x : \tau.\ t)\ u : B}\ \texttt{dc\_app\_beta}$$

Why they're the same: Both operations replace a redex with its contractum. The SC adds one successor edge in `cb_succ`. The proof graph adds one successor verified by `dr_cbn_once` (which wraps `dc_app_beta`). The correspondence is witnessed by the bisimulation: labels match (same terms), edges match (same successor), and the proof step is valid because β-reduction preserves typing.

Example 3: Case-split on neutral (synchronous choice) Supercompiler move:

Config: $\Gamma, l : \mathsf{List} \vdash \mathbf{case}\ l\ \mathbf{of}\ \{nil \Rightarrow b_0; cons \Rightarrow b_1\} : A$

Split step: `split_case_var` on neutral scrutinee l

Result: two configs

(nil branch) $\Gamma \vdash b_0 : A$

(cons branch) $\Gamma, x : \mathsf{Nat}, xs : \mathsf{List} \vdash b_1[(cons\ x\ xs)/l] : A[\ldots]$

Sequent proof step:

$$\frac{\Gamma \vdash b_0 : A \qquad \Gamma, x : \mathsf{Nat}, xs : \mathsf{List} \vdash b_1[(cons\ x\ xs)/l] : A[\ldots]}{\Gamma, l : \mathsf{List} \vdash \mathbf{case}\ l\ \mathbf{of}\ \{nil \Rightarrow b_0; cons \Rightarrow b_1\} : A}\ \texttt{dr\_split\_case\_var}$$

Why they're the same: SC's `split_case_var` computes the exact list of successor configurations (one per constructor), extending the context with constructor arguments and substituting the scrutinee. The proof rule `dr_split_case_var` computes the same list via `split_case_var_cfgs` (literally the same function). The SC graph has multiple outgoing edges in `cb_succ`; the proof graph has multiple premises. The correspondence is exact: `split_corresponds_to_sync_edge` proves that SC successors = proof premises.

Progress event: This is the only synchronous (choice-bearing) move. Both SC and the proof system mark this as a *progress event* in the trace condition: we've moved from analyzing an unknown list l to analyzing its constructor subcomponents (x, xs).

Example 4: Memo lookup / folding (backlink) Supercompiler move:

Current config: $\Gamma' \vdash t' : A'$

Memo lookup: finds earlier config v_{prev} with $\Gamma_0 \vdash t_0 : A_0$

Match: `judgement_eqb`$(t', t_0[\sigma])$ = `true` for some substitution σ

Result: SC sets `cb_succ`$[v_{curr}] := [v_{prev}]$ (backlink)

Sequent proof step:

$$\frac{\Gamma_0 \vdash t_0 : A_0 \qquad \Gamma' \vdash \sigma : \Gamma_0}{\Gamma' \vdash t_0[\sigma] : A_0[\sigma]}\ \texttt{nBack}$$

Why they're the same: The SC creates a cycle in the configuration graph by pointing the current node back to a previous node. The proof graph creates a cycle via a `nBack` node (backlink) with explicit substitution evidence σ. Both represent the same operation: "this goal is an instance of an earlier goal, so we close it by referring back." The correspondence `memo_corresponds_to_fold` proves that `judgement_eqb` soundly detects when this instantiation holds.

Cycle formation: This is where both artifacts become *cyclic*. The SC memo table prevents infinite unfolding; the proof backlink closes the proof with a cycle. Soundness requires the global trace condition (every cycle includes a progress event).

Example 5: Iota reduction (async driving) Supercompiler move:

Config: $\Gamma \vdash \mathbf{case}\ (cons\ h\ t)\ \mathbf{of}\ \{nil \Rightarrow b_0; cons\ x\ xs \Rightarrow b_1\} : A$

Drive step: `drive_cbn_once` performs iota (case-of-constructor)

Result: $\Gamma \vdash b_1[h/x, t/xs] : A$

Sequent proof step:

$$\frac{\Gamma \vdash b_1[h/x, t/xs] : A}{\Gamma \vdash \textbf{case}\ (\mathit{cons}\ h\ t)\ \textbf{of}\ \{\mathit{nil} \Rightarrow b_0; \mathit{cons}\ x\ xs \Rightarrow b_1\} : A}\ \texttt{dc\_case\_iota}$$

Why they're the same: When the scrutinee is a constructor (not a variable), SC immediately reduces to the corresponding branch. The proof system does the same via `dc_case_iota`. Both are deterministic (asynchronous) — there's no choice, just computation. The bisimulation is preserved: same term transformation, same single successor edge.

4.7 Bisimulation theorem

The above examples are instances of the general bisimulation. Formally:

Theorem 4.1 (Bisimulation (`bisim`)). *A supercompilation state* `scb : cfg_builder` *corresponds to a cyclic proof* `proof : rooted_preproof` *when:*

1. *Vertices match:* $\mathsf{dom}(\texttt{cb\_label}) = \mathsf{dom}(\texttt{pp\_label})$
2. *Labels match: for each vertex* v, $\texttt{cb\_label}(v) = \texttt{extract\_tm}(\texttt{pp\_label}(v))$
3. *Edges match: for each vertex* v, $\texttt{cb\_succ}(v) = \texttt{pp\_succ}(v)$
4. *Local validity: each proof vertex satisfies its sequent rule*

Theorem 4.2 (Main correspondence (`supercompile_gives_valid_preproof`)). *If* `supercompile_jTy` *succeeds on a typing judgement, then there exists a locally-valid* `rooted_preproof` *such that the bisimulation holds.*

Corollary: *Every SC operation (drive, split, fold) corresponds to a valid sequent proof operation, witnessed by* `drive_corresponds_to_async_edge`, `split_corresponds_to_sync_edge`, *and* `memo_corresponds_to_fold`.

These theorems are mechanised in `SupercompilationCorrespondence.v`.

4.8 Concrete example: `length (map f l)` fusion

To demonstrate the correspondence concretely, we trace the supercompilation of the classic fusion example:

$$\texttt{length}\ (\texttt{map}\ f\ l) \quad \text{where} \quad \Gamma = [l : \mathsf{List},\ f : \mathsf{Nat} \to \mathsf{Nat}]$$

This supercompiles to the same residual as `length` l, achieving full deforestation without any domain-specific rewrite rules.

Trace structure The supercompilation proceeds as follows (each step is a node/edge in both the SC graph and the cyclic proof):

1. **Unfold `map`**: Fix-unfolding (async edge, `dc_fix` rule)
2. **Split on** l: Case analysis on the list scrutinee (synchronous edge, `dr_split_case_var` rule). This is a *progress event* in the trace condition.
   - **Nil branch**: `length nil` $\rightsquigarrow$ `0` (direct reduction)
   - **Cons branch** (`cons` x xs):
     (a) `length (cons (f x) (map f xs))`

(b) Split on cons constructor: Iota reduction (async edge, `dc_case_iota` rule)

(c) Result: `succ (length (map f xs))`

(d) Recursive call: `length (map f xs)` (structurally smaller list xs)

3. **Fold**: Memo lookup finds the recursive call matches the original goal (modulo instantiation $l \mapsto xs$), creates a backlink (`nBack` node), forming a cycle.

Trace condition satisfied The cycle passes through two *progress edges* (splits on l and xs), each tracking a strictly smaller list by the recursive subterm ordering (constructor descent). This satisfies the ranking condition from `CyclicTraceCondition.v`.

Mechanisation The example is fully executable:

- Input terms: `t_len_map` and `t_len` in `CIUChecklistLengthMap.v`
- Supercompilation: `residualise_jTy 200 400 Σ Γ t_len_map nat_ty`
- Exact equality: `residualisation_length_map_exact` (proved by `vm_compute`)

This demonstrates the correspondence end-to-end: supercompilation moves are proof steps, and the result is a valid cyclic proof artifact.

5 Concrete Example: Length-Map Fusion

To illustrate the correspondence between supercompilation and cyclic proof search, we present a worked example: the classic deforestation problem of fusing `length` and `map`.

5.1 The input term

Consider the composed function that applies a function to every element of a list, then computes the length:

$$\lambda f.\, \lambda l.\, \texttt{length}\,(\texttt{map}\, f\, l)$$

This creates an intermediate list that is immediately consumed, which is inefficient. The expected optimized output is simply:

$$\lambda f.\, \lambda l.\, \texttt{length}\, l$$

5.2 Supercompilation trace

The supercompiler proceeds as follows:

1. **Initial configuration:** $\Gamma \vdash \lambda f.\, \lambda l.\, \texttt{length}\,(\texttt{map}\, f\, l) : (\texttt{Nat} \to \texttt{Nat}) \to \texttt{List} \to \texttt{Nat}$
2. **Drive (unfold):** Unfold `length` and `map` fixpoint definitions, exposing:

$$\lambda f.\, \lambda l.\, \texttt{case}\, l\, \texttt{of}\, \{\, \texttt{nil} \to \cdots \mid \texttt{cons}\, x\, xs \to \cdots \}$$

3. **Split (case analysis):** Split on the neutral variable l, creating two branches:
   - **Nil branch:** $\texttt{length}\,(\texttt{map}\, f\, \texttt{nil}) \rightsquigarrow 0$
   - **Cons branch:** $\texttt{length}\,(\texttt{map}\, f\, (\texttt{cons}\, x\, xs))$

4. **Drive (beta-iota):** In the cons branch, drive performs:

$$\texttt{length}\,(\texttt{cons}\,(f\,x)\,(\texttt{map}\,f\,xs)) \rightsquigarrow \texttt{succ}\,(\texttt{length}\,(\texttt{map}\,f\,xs))$$

5. **Fold (memoization):** Recognize that $\texttt{length}\,(\texttt{map}\,f\,xs)$ is a recursive call matching the initial configuration. Create a backlink to the root.

6. **Result:** The residual term is:

$$\lambda f.\,\lambda l.\,\texttt{case}\,l\,\texttt{of}\,\{\,\texttt{nil} \to 0 \mid \texttt{cons}\,x\,xs \to \texttt{succ}\,(\texttt{REC}\,f\,xs)\,\}$$

   where `REC` is the recursive call.

Note that the intermediate list construction `map f l` never appears in the residual term—it has been fused away.

5.3 Corresponding cyclic proof

The supercompilation trace above corresponds to a cyclic sequent proof with the following structure:

1. **Root node:** Sequent $\Gamma \vdash \lambda f.\,\lambda l.\,\texttt{length}\,(\texttt{map}\,f\,l) : (\texttt{Nat} \to \texttt{Nat}) \to \texttt{List} \to \texttt{Nat}$

2. **Asynchronous edges:** From unfolding `length` and `map`, each drive step creates a single successor with the driven term.

3. **Synchronous branching:** The case split on l creates two premises (nil and cons branches).

4. **Cyclic backlink:** The fold creates an edge back to the root node, forming a cycle.

5. **Local validity:** Each node satisfies its sequent rule:

   - Driving steps satisfy `dr_cbn_once`
   - The case split satisfies `dr_split_case_var`
   - The backlink is justified by configuration equality

5.4 Graph structure

The resulting proof graph has three nodes: the root (initial configuration), a nil branch returning 0, and a cons branch performing a recursive call. The cons branch has a backlink to the root, forming the cycle.

The cycle (from the cons branch back to root) represents the recursive structure. The *trace condition* for this cycle is satisfied because the case-split is a *progress point*: the recursive call is on xs, which is structurally smaller than l.

5.5 Mechanization and CIU validation

This example is mechanized in Coq in `theories/Transform/SupercompileTest.v`. We define the input term and run supercompilation with fuel parameter 20:

```
Definition length_map_input : tm :=
  tLam nat2nat (
    tLam list_ty (
      tApp length (tApp (tApp map (tVar 1)) (tVar 0)))).

Definition length_map_sc : option (nat * cfg_builder) :=
  supercompile_jTy 20 empty_env []
    length_map_input length_map_ty.
```

The supercompiler successfully processes this input, producing a `cfg_builder` with the structure described above. The correspondence theorems (Theorem 7.2) ensure that this configuration graph has a matching cyclic sequent proof.

5.6 Additional examples and CIU equivalence

The example suite in `Equiv/SupercompileChecklistIndexPipeline.v` validates Claim 4 of the pipeline on a range of standard supercompilation benchmarks. Each example runs through the supercompiler, and the residual is checked by computational reflection (`vm_compute`) against the expected result:

- **Length-map fusion:** `length` (`map` $f\ l$) residualises to a recursive function without intermediate list allocation. The residual is computationally equal to the expected deforested version.
- **Length-append normalisation:** `length` (`append` $l_1\ l_2$) produces a residual equal to `plus` (`length` l_1) (`length` l_2)
- **Append associativity:** `length` $((l_1 +\!\!+ l_2) +\!\!+ l_3)$ normalises via 2-pass supercompilation to the same canonical form as the right-associated variant.
- **Take-drop fusion:** `length` (`take` $n\ l +\!\!+$ `drop` $n\ l$) reduces to `length` l.
- **Map-map composition:** `length` (`map` f (`map` $g\ l$)) yields a non-trivial residual (further optimisation expected after strengthening the generalisation heuristic).
- **Vector index normalisation:** Residual indices from supercompiled vector expressions are equal by computation, demonstrating that the approach extends to dependent type indices.

All equalities are proved via `vm_compute` (no axioms), confirming that the supercompiler's output is definitionally equivalent to the expected optimised program for these benchmarks. The CIU equivalence theorem (Theorem 7.5) would generalise these computational equalities to the full contextual equivalence relation.

6 Related work

This section positions the present work relative to (i) supercompilation and program transformation, (ii) cyclic proof theory, (iii) sequent calculi for dependent types, and (iv) proof-theoretic treatments of induction.

6.1 Supercompilation

Supercompilation is a program transformation technique based on symbolic evaluation (driving), case analysis, generalization, and fold/refold steps [19, 16]. The key operations are:

- **Driving**: unfolding function definitions and performing symbolic reduction
- **Splitting**: case analysis on unknown values (neutrals)
- **Generalization**: abstracting common patterns to enable folding
- **Folding**: recognizing repeated configurations and creating recursive calls

Termination control. A central challenge in supercompilation is ensuring termination. Classical approaches use *whistles* based on homeomorphic embedding or well-quasi-orderings [8, 10, 17]. These approaches analyze configuration histories and force generalization when a dangerous pattern is detected. Our cyclic proof framework replaces the whistle with a *global trace condition*: we allow arbitrary folding but require that every cycle passes through a progress event (a split on a structurally smaller inductive component).

Supercompilation by evaluation. Recent work connects supercompilation to normalization-by-evaluation and partial evaluation [7]. Our contribution extends this connection to dependent types: we show that supercompilation *is* proof search in a typed setting, with configurations as sequents and the memo table as a cyclic proof graph.

Prior work on dependent types. To our knowledge, this is the first formulation of supercompilation for a dependently typed core language. Existing work on supercompilation focuses on first-order functional languages (Refal, Scheme, Haskell subsets) or simple typed settings [16, 7]. The dependent type context introduces new challenges:

- Types depend on terms, so driving must respect typing constraints
- The context Γ changes during proof search (extended in branches)
- Observational equivalence is more subtle (CIU for dependent types)

Our sequent calculus formulation addresses these challenges by making the typing context explicit in every sequent and using type-directed driving rules.

6.2 Cyclic proof theory

Cyclic proof systems represent potentially infinite derivations using finite directed graphs with back-edges, validated by a global soundness condition [3, 4].

Concrete cyclic systems. Brotherston and Simpson develop cyclic proofs for first-order logic with inductive definitions, using a global trace condition to ensure soundness. Their work establishes that cyclic proofs are as expressive as standard proofs with explicit induction rules, but offer computational advantages (the induction scheme emerges from the cycle structure).

Abstract cyclic frameworks. Afshari and Wehr develop a modern, highly abstract account of cyclic proof checking based on trace conditions, activation algebras, and categorical structure [1]. Our work instantiates their abstract framework in a concrete setting: CoC with inductives, CBN operational semantics, and observational equivalence.

Cyclic proofs as normal forms. Our contribution extends cyclic proof theory in a new direction: we use cyclic proofs not just as proof objects, but as *normal forms for program transformations*. The correspondence with supercompilation shows that cyclic proofs are stable under driving, splitting, and folding—operations that would be difficult to justify in a traditional natural deduction setting.

6.3 Sequent calculi for dependent types

Bidirectional type checking. Bidirectional type systems [14] separate synthesis (inferring types) from checking (validating against known types). This discipline is standard in dependently typed proof assistants (Agda, Coq, Lean). Our sequent calculus uses a similar decomposition, but with a crucial difference: we use *focused* sequents that separate asynchronous (deterministic) from synchronous (choice-bearing) phases.

Focused sequent calculi. Focusing [2, 11] is a proof search discipline that reduces nondeterminism by eagerly applying invertible rules (asynchronous phase) and carefully controlling choice points (synchronous phase). Focusing has been applied to linear logic, classical logic, and intuitionistic logic, but **to our knowledge this is the first focused sequent calculus for CoC with inductives**.

The key novelty is the connection to supercompilation: the asynchronous phase corresponds to driving (deterministic unfolding), and the synchronous phase corresponds to splitting and folding (the essential choice points).

Sequent calculi for CIC. Standard presentations of CIC use natural deduction [5, 12]. Herbelin develops a sequent calculus for the Calculus of Constructions [6], but without inductive types or cyclic backlinks. Our calculus extends Herbelin's work by adding:

- Inductive types and pattern matching
- Cyclic backlinks (folding)
- Focused phases (asynchronous vs synchronous)
- Connection to supercompilation

6.4 Induction: local rules vs global discharge

Traditional induction principles. In standard type theory, induction is a derived principle from the elimination rule for inductive types. To prove a property $P(x)$ for all $x : \mathsf{List}$, one applies the list eliminator (recursor) with a motive P and proofs for the base and step cases. This requires *choosing the induction principle upfront*—a significant practical burden.

Cyclic induction as post-hoc discovery. Sprenger and Dam compare local well-founded induction rules to cyclic systems with global discharge conditions [18]. They show these approaches are equivalent in expressive power but differ operationally: cyclic proofs discover the induction scheme during proof search rather than requiring it upfront.

Our work extends this insight to dependent types and connects it to supercompilation:

- The backlink corresponds to recognizing a recursive configuration
- The trace condition corresponds to structural descent (termination)
- The cycle structure *is* the induction principle, read off from the proof graph

This flexibility is essential for advanced optimizations (fusion, deforestation, tupling) where the appropriate induction principle is not obvious from the problem statement.

6.5 Proof-carrying code and certified compilation

Extracting programs from proofs. The Curry-Howard correspondence shows that proofs are programs and vice versa. Proof assistants like Coq use extraction to produce executable programs from constructive proofs [9]. Our work takes a different perspective: we view supercompilation as producing a *proof* (the cyclic derivation) from which the optimized program is extracted.

Translation validation vs proof transformation. Traditional certified compilers use translation validation: compile the program, then prove the output equivalent to the input [15]. Our approach is proof transformation: the supercompiler operates on proof graphs, and soundness follows from preservation of cyclic structure. This is similar in spirit to recent work on intrinsically typed compilation [13].

6.6 Summary

This work makes several novel contributions:

1. **First supercompilation for dependent types**, addressing challenges of dependent contexts and type-directed transformation
2. **First focused sequent calculus for CoC + inductives**, cleanly separating deterministic driving from choice points
3. **Exact operational correspondence** between supercompilation and cyclic proof search, established via bisimulation and mechanized in Coq
4. **Post-hoc induction principle discovery**, showing how cyclic proofs enable flexible recursion schemes

The framework provides a principled foundation for program optimization in dependently typed languages and opens new avenues for proof-directed compilation.

7 Main results: the four-claim pipeline

We organise the central results as a pipeline of four claims, each building on the previous. Claim 1 asserts that supercompilation always produces a locally-valid pre-proof graph. Claim 2 shows that when the trace-condition check is enabled, the graph upgrades to a ranked cyclic proof. Claim 3 conjectures that cyclic proof soundness implies CIU equivalence of the residualised program. Claim 4 validates the pipeline on concrete examples.

7.1 Claim 1: Supercompilation yields a pre-proof

Theorem 7.1 (Pre-proof correspondence)**.** *For any well-formed environment Σ, fuel f, context Γ, term t, and type A, if*

$$\texttt{supercompile\_jTy}\ f\ \Sigma\ \Gamma\ t\ A\ =\ \mathsf{Some}\,(v, scb)$$

then scb directly forms a rooted_preproof *with root v. Every vertex is labelled and satisfies its local sequent rule; the edge structure corresponds exactly to the drive, split, and fold operations of the supercompiler.*

This claim is mechanized in Coq via `all_supercompiled_programs_yield_preproof` (`SupercompilationCorre`
The step-level correspondence is given by three graph-level theorems:

Theorem 7.2 (Step correspondence (drive/split/fold))**.** *Each atomic supercompilation move corresponds to exactly one local inference step in the proof graph:*

1. ***Driving (asynchronous):*** *If the supercompiler performs one-step CBN reduction on a configuration at vertex v, producing a single successor at w, then the proof graph has edge $v \to w$ and w carries the driven configuration. (Coq: `drive_corresponds_to_async_edge`, `SupercompilationCorrespondence.v:465`.)*

2. ***Splitting (synchronous):*** *If the supercompiler splits on a neutral case-variable at v, producing n successors $w_1, \ldots, w_n$, then the proof graph has edges $v \to w_i$ and each w_i carries the branch configuration. (Coq: `split_corresponds_to_sync_edge`, `SupercompilationCorrespondence.v:513`.)*

3. ***Folding (backlink):*** *If the supercompiler performs a memo hit, creating a backlink $v \to v_{prev}$, then the proof graph has the backlink edge and both vertices have matching labels. (Coq: `memo_corresponds_to_fold`, `SupercompilationCorrespondence.v:581`.)*

Remark 7.3. *The proofs rely on a* bisimulation *relation between the SC configuration graph (`cfg_builder`) and the cyclic proof graph (`rooted_preproof`). The bisimulation ensures: (i) same vertices, (ii) matching labels (SC configurations = proof sequents), (iii) matching edges, and (iv) local validity. See Section 8.*

7.2 Claim 2: Termination yields a cyclic proof

Theorem 7.4 (Cyclic proof via budget trace)**.** *If the supercompiler succeeds with the trace-condition check enabled:*

$$\texttt{supercompile\_jTy\_tc}\ f\ \Sigma\ \Gamma\ t\ A\ =\ \mathsf{Some}\,(v, scb)$$

then there exists a Ranked.rooted_cyclic_proof *with root vertex (v, B) on the budget-instrumented trace graph, where B is the number of labelled vertices in scb. The ranking witness uses the natural numbers with standard less-than ordering, and the rank of each trace vertex (v, k) is the remaining budget k.*

This claim is mechanized via `supercompile_yields_rooted_cyclic_proof` (`SupercompilationCorresponden`
The construction proceeds in two stages:

1. **Cycle-progress:** The boolean trace check `SC.trace_condition_ok` guarantees that every directed cycle in the configuration graph contains a progress edge—a case-split on a neutral scrutinee. (Coq: `trace_condition_ok_cycle_progress`, `SupercompileTraceCheckSound.v:610`.)

2. **Budget trace ranking:** Given cycle-progress, we apply the general budget-trace construction from `CyclicTraceConditionBudget.v`. The base graph is the configuration graph; the base vertices are lifted to pairs (v, k) where $k \in [0, B]$ is a budget counter. Non-progress edges preserve k; progress edges consume one unit. The resulting trace graph is acyclic, yielding a full ranking_condition with rank $\lambda(v, k).\, k$. Each trace vertex is labelled with the judgement of its base vertex, giving a pre-proof on the trace graph. Combined with the ranking, this forms a Ranked.cyclic_proof.

7.3 Claim 3: CIU soundness

Theorem 7.5 (CIU soundness for supercompilation)**.** *Let Σ be a well-formed environment, Γ a context, $t : A$ a well-typed term, and scb the configuration graph produced by successful supercompilation of t with the trace check. Then*

$$t \approx_{\text{ciu}} \texttt{residualise\_cfg}(scb)$$

where $\approx_{\text{ciu}}$ is the untyped *CIU equivalence: for every closing substitution σ (not necessarily typed) and every CBN value v, we have* `terminates_to` $t[\sigma]\ v$ *iff* `terminates_to` $t_{res}[\sigma]\ v$.

The typed variant $\approx_{\text{ciu\_jTy}}$ follows directly: since the untyped CIU quantifies over all substitutions, instantiating with the list-based substitution used by ***`ciu_jTy`*** *gives the typed result (Coq:* ***`supercompile_ciu_soundness_typed`****, 8 lines).*

This claim is mechanized via `supercompile_ciu_soundness_untyped` (`SupercompilationCorrespondence.v`), proved by induction on fuel with the trace condition ensuring that self-loops—which would introduce non-termination—are rejected. The proof structure:

1. **Step CIU (CIU.v):** Every CBN reduction step preserves CIU (`step_ciu`). Every supercompiler driving step composes to a chain of CBN steps (`drive_cbn_once_ciu`).

2. **Constructor CIU (CIU.v):** CIU lifts through term constructors: `ciu_tApp`, `ciu_apps`, `ciu_shift`, `ciu_subst0`, `ciu_case_branches_ciu`.

3. **Fuel induction:** `residualise_cfg_ciu` proves by induction on fuel that for any vertex v not yet in the fix environment, the residualised term is CIU-equivalent to the shifted term at v. The `tFix` wrapper is handled by `ciu_fix` and the `subst0_shift_cancel` lemma which cancels one level of de Bruijn shift.

4. **Self-loop protection:** The trace check (`trace_condition_ok`) prevents self-loops without progress edges (`trace_condition_ok_no_self_loop`), guaranteeing that the fuel induction never encounters a degenerate fixpoint that would introduce non-termination.

5. **Generalisation:** The `lookup_inst` case (generalisation) is handled by `ciu_generalise`: `apps (mk_lams tys body) args` β-reduces to the body with arguments substituted, filling the anti-unification holes. The induction on the generalised vertex then gives CIU for the original vertex via the instantiation substitution.

The CIU relation itself is defined and proved to be an equivalence in Coq (`Equiv/CIU.v`); the `terminates_to_tApp_decompose` and `terminates_to_tCase_decompose` lemmas in `Semantics/Cbn.v` provide the decomposition needed for the constructor-lifting proofs.

7.4 Claim 4: Example validation

Proposition 7.6 (Example validation)**.** *The following supercompilation examples satisfy the pipeline and produce residuals that are computationally equivalent to the hand-optimised target programs:*

- $\mathsf{length}\,(\mathsf{map}\,f\,l)$ — *deforested to an equivalent recursive function without intermediate list construction.*
- $\mathsf{length}\,(\mathsf{append}\,l_1\,l_2)$ — *fused with* $\mathsf{plus}\,(\mathsf{length}\,l_1)\,(\mathsf{length}\,l_2)$.
- $\mathsf{length}\,((l_1 +\!\!+ l_2) +\!\!+ l_3)$ — *normalised via 2-pass supercompilation to a canonical form matching the right-associated variant.*
- $\mathsf{length}\,(\mathsf{take}\,n\,l +\!\!+ \mathsf{drop}\,n\,l)$ — *reduces to* $\mathsf{length}\,l$.

All equalities are verified by Coq's `vm_compute` *in* ***`Equiv/SupercompileChecklistIndexPipeline.v`****.*

Additional examples include $\mathsf{length}\,(\mathsf{map}\,f\,(\mathsf{map}\,g\,l))$ (supercompiler produces a non-trivial residual, with further optimisation expected after strengthening the generalisation heuristic), and vector-index normalisation (residuals yield equal indices after supercompilation).

8 Mechanization

The four-claim pipeline is mechanized in Coq (Rocq 9.1.0) using the `stdpp` library for finite maps and graphs. The development totals ∼5,000 lines of proof across 14 files. All theorems below are proved with no axioms or admitted lemmas.

8.1 Claim 1: Pre-proof correspondence

Theorem 8.1 (Pre-proof construction)**.** *For any environment* Σ*, fuel* f*, context* Γ*, term* t*, and type* A*, if* `supercompile_jTy` $f\ \Sigma\ \Gamma\ t\ A = \mathsf{Some}\,(v, scb)$*, then* scb *directly forms a* rooted_preproof *with root* v*.*

Coq: `all_supercompiled_programs_yield_preproof` (`SupercompilationCorrespondence.v:1659`).

Theorem 8.2 (Step correspondence)**.** *Each supercompilation edge corresponds to a sequent rule.*
1. ***Driving (async):*** *`drive_corresponds_to_async_edge` (line 465)*
2. ***Splitting (sync):*** *`split_corresponds_to_sync_edge` (line 513)*
3. ***Folding (backlink):*** *`memo_corresponds_to_fold` (line 581)*

Proposition 8.3 (Vertex rule classification)**.** *Every cfg_builder vertex satisfies a specific drive rule determined by its successor structure: 0 successors ⇒ leaf; 1 successor with driving ⇒ async; 1 successor without change ⇒ fold;* $n > 1$ *successors ⇒ split.*
Coq: ***`cfg_vertex_drive_rule (SupercompilationCorrespondence.v)`****.*

8.2 Claim 2: Cyclic proof via budget trace

Theorem 8.4 (Cyclic proof from trace check)**.** *If the supercompiler succeeds with the trace-condition check,* `supercompile_jTy_tc` $f\ \Sigma\ \Gamma\ t\ A = \mathsf{Some}\,(v, scb)$*, then there exists a* Ranked.rooted_cyclic_proof *on a budget-instrumented trace graph with budget* $B = |\mathsf{dom}(scb.\mathsf{cb\_label})|$ *and rank* $\lambda(v, k).\, k$ *ordered by* $<_{\mathbb{N}}$*.*

Coq: `supercompile_yields_rooted_cyclic_proof` (`SupercompilationCorrespondence.v`).

The construction (∼140 lines) instantiates the general budget-trace theorem from `CyclicTraceConditionBudget`

1. The base graph is cfg_graph(scb), with progress predicate `is_progress_vertex`.
2. The trace graph lifts each base vertex v to pairs (v, k) for $0 \le k \le B$. Progress edges consume one unit of budget; non-progress edges preserve it.
3. Pigeonhole argument: cycle-progress implies the trace graph is acyclic, yielding a full ranking_condition.
4. Labelling each trace vertex with the judgement of its base vertex (permissive rule) gives a preproof on the trace graph.

8.3 Claim 3: CIU soundness

Theorem 8.5 (CIU soundness, untyped)**.** *If* `supercompile_jTy_tc` *succeeds on t, the residualised term is CIU-equivalent to t:*

$$\forall \sigma, v.\ \texttt{terminates\_to}\ t[\sigma]\ v \iff \texttt{terminates\_to}\ t_{res}[\sigma]\ v$$

Coq: `supercompile_ciu_soundness_untyped` (`SupercompilationCorrespondence.v`).

The proof is by induction on the fuel parameter of the residualiser, using the following component lemmas (all proved):

- `step_ciu`, `steps_ciu`: any CBN step preserves CIU (`Equiv/CIU.v`)
- `ciu_tApp`, `ciu_apps`, `ciu_shift`, `ciu_subst0`, `ciu_case_branches_ciu`: CIU lifts through all term constructors (`Equiv/CIU.v`)
- `terminates_to_tApp_decompose`, `terminates_to_tCase_decompose`: CBN decomposition lemmas (`Semantics/Cbn.v`)
- `drive_cbn_once_ciu`, `whnf_drive_ciu`: supercompiler driving preserves CIU (`SupercompilationCorresponde`
- `ciu_fix`, `ciu_fix_nonrec`: fixpoint unrolling preserves CIU (`Equiv/CIU.v`)
- `trace_condition_ok_no_self_loop`: the trace check blocks cycles without progress (`SupercompileTraceCheck`

The typed variant `supercompile_ciu_soundness_typed` follows in 8 lines: the untyped CIU quantifies over all substitutions, and the typed restriction instantiates the list-based substitution of `ciu_jTy`.

Theorem 8.6 (Typing preservation for leaf vertices)**.** *For a leaf vertex (no successors) with label* $\mathsf{jTy}(\Gamma, t, A)$ *and adequate fuel, the residual has type A in context Γ.*
Coq: `residualise_cfg_root_typing` (`SupercompilationCorrespondence.v`).

The typing proof uses `has_type_subst` (renaming preserves typing, all 9 rules), `has_type_weaken_head` (weakening by one binder), and `has_type_shift` (uniform shift preserves typing) — all proved in `Judgement/Typing.v`.

8.4 Claim 4: Example validation

The example suite in `Equiv/SupercompileChecklistIndexPipeline.v` covers length-map fusion, append-length normalisation, append associativity (2-pass), take-drop fusion, map-map composition, and vector-index normalisation. All equalities are verified by computational reflection (`vm_compute`) with no axioms.

8.5 Summary of Coq artefacts

Component	Key lemmas	File
Syntax & semantics	term, CBN evaluation	`Syntax/Term.v`, `Semantics/Cbn.v`
Typing	`has_type`, `has_type_subst`	`Judgement/Typing.v`
Graph theory	`fin_digraph`, `ranking_condition`	`Graph/FiniteDigraph.v`, `Progress/Ranking.v`
Pre-proofs & cyclic proofs	`preproof`, `cyclic_proof`	`Preproof/Preproof.v`, `CyclicProof/`
Supercompiler	`supercompile_cfg`, `residualise_cfg`	`Transform/Supercompile.v`
Trace check soundness	`trace_condition_ok_cycle_progress`	`SupercompileTraceCheckSound.v`
Budget trace ranking	`budget_trace_ranking_condition`	`CyclicTraceConditionBudget.v`
All pipeline theorems: `SupercompilationCorrespondence.v`		
Claim 1	`all_supercompiled_programs_yield_preproof`, `drive/split/fold_corresponds_to_edge`, `cfg_vertex_drive_rule`	
Claim 2	`supercompile_yields_rooted_cyclic_proof`	
Claim 3	`supercompile_ciu_soundness_untyped/typed`, `residualise_cfg_root_typing`	
Claim 4	`CIUChecklistLengthMap`, `SupercompileChecklistIndexPipeline`	`Equiv/Equiv/`

All proofs are closed (`Qed`); the development contains zero axioms or admitted lemmas.

9 Conclusion

This paper develops a focused cyclic sequent proof-search view of supercompilation for a dependently typed calculus (CoC with inductives). The central claim is operational: supercompilation moves (drive/split/fold) coincide with local inference steps in a cyclic proof graph, and the cyclic global progress condition matches the structural-descent termination discipline that justifies folding.

Contributions. We organise the results as a four-claim pipeline, each mechanized in Coq:

1. **Pre-proof construction (proved):** Supercompilation success always yields a locally-valid rooted pre-proof whose edge structure corresponds exactly to drive/split/fold operations.
2. **Cyclic proof via budget trace (proved):** With the trace-condition check enabled, the pre-proof upgrades to a ranked cyclic proof on a budget-instrumented trace graph.
3. **CIU soundness (proved):** The residualised program is CIU-equivalent to the original source term. The proof uses fuel induction with the trace condition preventing self-loops that would introduce non-termination. Supporting lemmas lift CIU through all term constructors (`ciu_tApp`, `ciu_apps`, `ciu_shift`, `ciu_subst0`, `ciu_case_branches_ciu`).
4. **Example validation (demonstrated):** Concrete supercompilation examples (length-map fusion, append length, take-drop, etc.) pass the pipeline with computational equivalence verified by Coq's `vm_compute`.

Limitations. The current presentation focuses on a sequent system tailored to the supercompilation correspondence rather than a full CIC sequent calculus; several CIC-specific features (conversion, universe cumulativity, dependent elimination obligations) are deferred. The CIU proof handles the generalisation-free fragment (no instantiation of generalised configurations); the generalisation case requires extending the CIU lifting lemmas to the `apps`-with-substitution pattern.

Future work. Immediate next steps are: (i) extending the CIU proof to cover the generalisation operation, (ii) extending the focused calculus to cover a fuller CIC sequent system, and (iii) strengthening the generalisation heuristic (whistle) to optimise more complex patterns such as nested deforestation. We also view oracle-guided generalisation (including LLM-generated candidates) as promising future work. The oracle proposes generalisation candidates while the kernel performs deterministic validation checks; this shifts the problem from enumeration to certification. Since all soundness-critical obligations are discharged by the kernel (rule checking, typing of substitution evidence, global progress validation, CIU preservation), the oracle is not trusted for soundness.

References

[1] Bahareh Afshari and Dominik Wehr. Abstract cyclic proofs. *Mathematical Structures in Computer Science*, 34:552–577, 2024.

[2] Jean-Marc Andreoli. Logic programming with focusing proofs in linear logic. *Journal of Logic and Computation*, 2(3):297–347, 1992.

[3] James Brotherston. Cyclic proofs for first-order logic with inductive definitions. In *TABLEAUX*, 2006.

[4] James Brotherston and Alex Simpson. Sequent calculi for induction and infinite descent. In *LICS*, 2011.

[5] Thierry Coquand and Gérard Huet. The calculus of constructions. *Information and Computation*, 76(2–3):95–120, 1988.

[6] Hugo Herbelin. *A Lambda-Calculus Structure Isomorphic to Gentzen-Style Sequent Calculus Structure*. PhD thesis, University of Paris 7, 1995.

[7] Neil D. Jones, Carsten K. Gomard, and Peter Sestoft. *Partial Evaluation and Automatic Program Generation*. Prentice Hall, 1993. Classic textbook on partial evaluation and metaprogramming.

[8] Joseph B. Kruskal. Well-quasi-ordering, the tree theorem, and Vazsonyi's conjecture. *Transactions of the American Mathematical Society*, 1960.

[9] Pierre Letouzey. A new extraction for Coq. In *Types for Proofs and Programs (TYPES 2002)*, pages 200–219. Springer, 2003.

[10] Michael Leuschel. Homeomorphic embedding for online termination of partial deduction. In *Logic-Based Program Synthesis and Transformation*, 2002.

[11] Dale Miller and Alexis Saurin. From proofs to focused proofs: A modular proof of focalization in linear logic. In *Computer Science Logic (CSL)*, pages 405–419. Springer, 2004.

[12] Christine Paulin-Mohring. Inductive definitions in the system Coq: Rules and properties. In *Typed Lambda Calculi and Applications*, pages 328–345. Springer, 1993.

[13] Patrick Bahr Pickard, Graham Hutton, and Jeremy Gibbons. Calculating correct compilers. *Journal of Functional Programming*, 22:1–35, 2012. Presents calculation-based approach to compiler correctness.

[14] Benjamin C. Pierce and David N. Turner. Local type inference. In *ACM Transactions on Programming Languages and Systems (TOPLAS)*, volume 22, pages 1–44. ACM, 2000.

[15] Amir Pnueli, Michael Siegel, and Eli Singerman. Translation validation. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, pages 151–166. Springer, 1998.

[16] Morten Heine Sørensen and Robert Glück. An introduction to supercompilation. In *Partial Evaluation and Semantics-Based Program Manipulation*, 1995. Often cited introductory survey; venue details vary by edition.

[17] Morten Heine Sørensen, Robert Glück, and Neil D. Jones. A positive supercompiler. In *Journal of Functional Programming*, volume 6, pages 811–838. Cambridge University Press, 1996.

[18] Christoph Sprenger and Mads Dam. On the structure of inductive reasoning: Circular and tree-shaped proofs in the μ-calculus. In *Foundations of Software Science and Computation Structures (FoSSaCS 2003)*, volume 2620 of *Lecture Notes in Computer Science*, pages 425–440. Springer, 2003.

[19] Valentin F. Turchin. *The Concept of a Supercompiler*. Springer, 1986. Classic reference introducing supercompilation.